

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторные работы №1–3
По дисциплине «Аппаратное и программное обеспечение сетей»

Тема: «TCP- и UDP-приложения на Java»

Выполнил:

Студентка ФЭИС

Группы ИИ-27

Киричук В.А.

Проверил:

Кулик А.Д.

Брест 2026

Лабораторная работа №1. Организация TCP-сервера

Цель работы: Изучить основы программирования сетевых приложений на языке Java с использованием пакета `java.net`, классы `ServerSocket` и `Socket`; разработать многопоточный TCP-эхо-сервер в соответствии с индивидуальным заданием (вариант 4).

Индивидуальное задание (вариант 4):

Сервер (задание №4)	После приёма каждой группы из 64 символов от клиента сервер формирует и отправляет цепочку <«символ1-количество1», «символ2-количество2», ...>, где «символ» – встреченный в группе символ, «количество» – число его вхождений в группу. Если в очередной группе число различных символов меньше 3, клиенту отправляется соответствующее сообщение и сервер разрывает соединение.
Клиент	Программа <code>telnet</code> (адрес 127.0.0.1, порт сервера 8080) согласно п. 8.2 методических указаний.

Краткие теоретические сведения:

TCP (Transmission Control Protocol) – протокол транспортного уровня с установлением соединения. Он гарантирует доставку данных, повторную передачу потерянных пакетов и получение байт в том порядке, в котором они были отправлены; за это приходится платить повышенными накладными расходами. Соединение однозначно определяется четырьмя значениями: адрес и порт клиента, адрес и порт сервера.

В пакете `java.net` для TCP предназначены два класса. `ServerSocket` – серверный сокет: создаётся с указанием порта, метод `accept()` блокирует программу до подключения клиента и возвращает объект `Socket`, представляющий это соединение. `Socket` – клиентский сокет (или соединение, полученное сервером): из него через `getInputStream()` и `getOutputStream()` получают потоки ввода-вывода, которые оборачивают в `Reader/Writer` (`InputStreamReader`, `BufferedReader`, `PrintWriter`). Буфер вывода нужно сбрасывать (`flush`), иначе данные не будут отправлены.

Сокеты занимают системные ресурсы, поэтому их закрывают в блоке `finally`.

Чтобы сервер мог общаться с несколькими клиентами одновременно, для каждого принятого соединения создаётся отдельный поток (`Thread`), а основной поток снова вызывает `accept()`.

В качестве клиента по заданию используется программа `telnet`. Она посылает символы по мере набора, а при подключении отправляет служебные команды протокола `telnet` (последовательности, начинающиеся с байта 255). Сервер такие команды отбрасывает и не считает их символами группы. `Telnet` по умолчанию не показывает набранный текст сам, поэтому сервер возвращает (эхо) каждый принятый символ.

Ход работы:

Сервер принимает соединения на порту 8080. Для каждого клиента запускается поток `ServeOne`. Поток читает символы, возвращает их клиенту (эхо), накапливает в группу и после 64-го символа подсчитывает число вхождений каждого символа. Символы конца строки в группу не входят. Если различных символов в группе меньше 3, клиенту отправляется сообщение, после чего сокет закрывается.

Ниже приведены текст программы с комментариями и результаты проверки.

Листинг программы `TCPServer.java`:

```
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.LinkedHashMap;
import java.util.Map;
```

```

/**
 * Лабораторная работа №1. TCP-эхо-сервер.
 * Вариант 4: задание сервера №4.
 *
 * Сервер принимает поток символов от клиента (клиент - telnet) и возвращает его
 * обратно (эхо). После приёма каждой группы из 64 символов он формирует и отправляет
 * цепочку <'символ1'-количество1, 'символ2'-количество2, ...>, где "символ" -
 * встреченный в группе символ, "количество" - число его вхождений в группе.
 * Если в группе различных символов меньше 3, сервер отправляет сообщение и
 * разрывает соединение.
 *
 * Для одновременного обслуживания нескольких клиентов на каждого клиента
 * создаётся отдельный поток (класс ServeOne), как в примере MultiServer из методички.
 */
public class TCPServer {

    public static int PORT = 8080;
    static final int GROUP = 64;           // размер группы символов
    static final int MIN_DISTINCT = 3;     // минимум различных символов в группе

    public static void main(String[] args) throws IOException {
        if (args.length > 0) {
            PORT = Integer.parseInt(args[0]);
        }
        ServerSocket s = new ServerSocket(PORT);
        log("Сервер запущен. Порт " + PORT);
        try {
            while (true) {
                // Останавливает выполнение, пока не придёт новое соединение.
                Socket socket = s.accept();
                try {
                    new ServeOne(socket);           // поток обслуживает этого клиента
                } catch (IOException e) {
                    // Если не удалось - закрываем сокет, иначе его закроет поток.
                    socket.close();
                }
            }
        } finally {
            s.close();
        }
    }

    static void log(String msg) {
        System.out.println(new SimpleDateFormat("HH:mm:ss.SSS").format(new Date()) + "
" + msg);
    }

    /** Поток, обслуживающий одного клиента. */
    static class ServeOne extends Thread {
        private final Socket socket;
        private final Reader in;
        private final Writer out;
        private final String client;

        ServeOne(Socket s) throws IOException {
            socket = s;
            client = s.getInetAddress().getHostAddress() + ":" + s.getPort();
            // Входной поток байт -> фильтр служебных команд telnet -> символы UTF-8.
            in = new InputStreamReader(new TelnetFilter(s.getInputStream()),
StandardCharsets.UTF_8);
            out = new BufferedWriter(new OutputStreamWriter(s.getOutputStream(),
StandardCharsets.UTF_8));
            start();                               // вызывает run()
        }

        /** Отправка строки клиенту. telnet ожидает конец строки "\r\n". */
        private void send(String text) throws IOException {
            out.write(text);
            out.flush();                             // без сброса буфера данные не уйдут
в сеть
        }
    }
}

```

```

@Override
public void run() {
    log("Подключился клиент " + client);
    try {
        send("Hello, Student!\r\n");
        StringBuilder group = new StringBuilder();    // текущая группа символов
        int c, prev = 0;
        while ((c = in.read()) != -1) {
            char ch = (char) c;
            // Эхо: возвращаем клиенту всё, что он набрал (Enter -> перевод
строки) .

            if (ch == '\r' || (ch == '\n' && prev != '\r')) {
                send("\r\n");
            } else if (ch != '\n' && ch != '\0') {
                send(String.valueOf(ch));
            }
            prev = ch;
            // Символы конца строки в группу не входят.
            if (ch == '\r' || ch == '\n' || ch == '\0') continue;
            group.append(ch);
            if (group.length() < GROUP) continue;

            // Набрана полная группа из 64 символов - считаем статистику.
            Map<Character, Integer> counts = new LinkedHashMap<>();
            for (int i = 0; i < group.length(); i++) {
                counts.merge(group.charAt(i), 1, Integer::sum);
            }
            group.setLength(0);
            if (counts.size() < MIN_DISTINCT) {
                log("В группе " + counts.size() + " различных символов. Разрыв
соединения с " + client);
                send("\r\nВ группе из " + GROUP + " символов меньше " +
MIN_DISTINCT
                    + " различных (найдено " + counts.size() + ").
Соединение разорвано.\r\n");
                break;    // выходим из цикла -> finally
закроет сокет
            }
            StringBuilder chain = new StringBuilder("<");
            for (Map.Entry<Character, Integer> e : counts.entrySet()) {
                if (chain.length() > 1) chain.append(", ");
                chain.append('\''').append(e.getKey()).append("' -
").append(e.getValue());
            }
            chain.append('>');
            log("Группа от " + client + ": " + chain);
            send("\r\n" + chain + "\r\n");
        }
    } catch (IOException e) {
        log("Ошибка связи с " + client + ": " + e.getMessage());
    } finally {
        // Сокет закрываем всегда, независимо от того, было исключение или нет.
        try {
            socket.close();
        } catch (IOException e) {
            log("Сокет не закрыт");
        }
        log("Соединение с " + client + " закрыто");
    }
}

/**
 * Фильтр входного потока: вырезает служебные команды протокола telnet
 * (последовательности, начинающиеся с байта 255 = IAC), которые клиент
 * telnet.exe отправляет при подключении. Остальные байты пропускает как есть.
 */
static class TelnetFilter extends InputStream {
    private final InputStream in;

    TelnetFilter(InputStream in) {
        this.in = in;
    }
}

```

```

    }

    @Override
    public int read() throws IOException {
        int b = in.read();
        if (b != 255) return b;                // обычный байт
        int cmd = in.read();
        if (cmd == -1) return -1;
        if (cmd == 255) return 255;            // IAC IAC - настоящий байт 255
        if (cmd >= 251 && cmd <= 254) {        // WILL/WONT/DO/DONT + код опции
            in.read();
        } else if (cmd == 250) {               // SB ... IAC SE (подопции)
            int prev = 0, x;
            while ((x = in.read()) != -1) {
                if (prev == 255 && x == 240) break;
                prev = x;
            }
        }
        return read();                        // идём дальше к следующему байту
    }

    /** Не ждём заполнения всего буфера: возвращаем то, что уже пришло. */
    @Override
    public int read(byte[] buf, int off, int len) throws IOException {
        int first = read();
        if (first == -1) return -1;
        buf[off] = (byte) first;
        int n = 1;
        while (n < len && in.available() > 0) {
            int x = read();
            if (x == -1) break;
            buf[off + n++] = (byte) x;
        }
        return n;
    }
}
}

```

Проверка выполнена по методике из раздела 7.1 методических указаний: клиент подключается к 127.0.0.1:8080, видит приветствие «Hello, Student!», набирает текст и получает его обратно. Последовательность: короткая строка «hello udf» (9 символов); строка из 55 символов, дополняющая группу до 64 (сервер присылает статистику); 64 символа «abab...» с двумя различными символами (сервер разрывает соединение).

ab
В группе из 64 символов меньше 3 различных (найдено 2). Соединение разорвано.
<<closed>>

Рисунок 1 – Сеанс работы клиента с сервером

```
23:47:50.714 Сервер запущен. Порт 8080
23:47:52.098 Подключился клиент 127.0.0.1:38532
23:47:53.567 Группа от 127.0.0.1:38532: <'h'-3, 'e'-4, 'l'-3, 'o'-5, ' '-12, 'u'-4, 'd'-3,
'p'-2, 't'-2, 'q'-1, 'i'-1, 'c'-1, 'k'-1, 'b'-1, 'r'-3, 'w'-1, 'n'-3, 'f'-2, 'x'-1, 'j'-1,
'm'-1, 's'-2, 'v'-1, 'a'-3, 'z'-1, 'y'-1, 'g'-1>
23:47:54.650 В группе 2 различных символов. Разрыв соединения с 127.0.0.1:38532
23:47:54.652 Соединение с 127.0.0.1:38532 закрыто
```

Рисунок 2 – Консоль ТСР-сервера

Вывод: В ходе работы изучены классы `ServerSocket` и `Socket` пакета `java.net`. Разработан многопоточный ТСР-эхо-сервер, который принимает поток символов, возвращает его клиенту, после каждой группы из 64 символов присылает статистику «символ-количество» и разрывает соединение при числе различных символов в группе меньше 3. Корректность работы подтверждена проверкой на `127.0.0.1:8080`; служебные команды `telnet` сервером отбрасываются.

Лабораторная работа №2. Организация TCP-клиента

Цель работы: Изучить классы пакета `java.net` для написания TCP-клиента; разработать TCP-клиент в соответствии с индивидуальным заданием (вариант 4) и проверить его работу с TCP-сервером из лабораторной работы №1.

Индивидуальное задание (вариант 4):

Клиент (задание №5)	Ввод символов с отсылкой введенной строки по нажатию клавиши Home. Ведение файла протокола событий: 1) время начала и окончания соединения; 3) принимаемая от сервера строка и время приёма. Разрыв соединения командой <code>disconnect <адрес> <порт></code> . Подключение к серверу специальной командой <code>connect <адрес> <порт></code> .
Сервер	TCP-сервер, разработанный в лабораторной работе №1 (адрес 127.0.0.1, порт 8080).

Краткие теоретические сведения:

TCP (Transmission Control Protocol) – протокол транспортного уровня с установлением соединения. Он гарантирует доставку данных, повторную передачу потерянных пакетов и получение байт в том порядке, в котором они были отправлены; за это приходится платить повышенными накладными расходами. Соединение однозначно определяется четырьмя значениями: адрес и порт клиента, адрес и порт сервера.

В пакете `java.net` для TCP предназначены два класса. `ServerSocket` – серверный сокет: создаётся с указанием порта, метод `accept()` блокирует программу до подключения клиента и возвращает объект `Socket`, представляющий это соединение. `Socket` – клиентский сокет (или соединение, полученное сервером): из него через `getInputStream()` и `getOutputStream()` получают потоки ввода-вывода, которые оборачивают в `Reader/Writer` (`InputStreamReader`, `BufferedReader`, `PrintWriter`). Буфер вывода нужно сбрасывать (`flush`), иначе данные не будут отправлены. Сокеты занимают системные ресурсы, поэтому их закрывают в блоке `finally`.

Клиент создаёт `Socket`, указав адрес и порт сервера; свой порт клиенту назначает система. Приём данных от сервера выполняется в отдельном потоке, чтобы ожидание ответа не блокировало интерфейс пользователя.

Ход работы:

Клиент реализован как окно Swing: сверху журнал обмена, снизу строка ввода. Нажатие клавиши Home отправляет введенную строку серверу (обработчик `KeyListener`, код `VK_HOME`). Подключение выполняется командой `connect <адрес> <порт>`, разрыв соединения – командой `disconnect <адрес> <порт>`. Файл протокола `tcp_client.log` содержит время начала и окончания соединения и каждую принятую от сервера строку с временем приёма. Если окружение без графики или указан ключ `--console`, клиент работает в консоли.

Листинг программы `TCPClient.java`:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.io.*;
import java.net.InetAddress;
import java.net.Socket;
import java.nio.charset.StandardCharsets;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.function.Consumer;

/**
 * Лабораторная работа №2. TCP-клиент.
```

```

* Вариант 4: задание клиента №5 (табл. 3):
*   - ввод символов с отправкой введённой строки по нажатию клавиши Home;
*   - ведение файла протокола событий tcp_client.log, включающего:
*       1) время начала и окончания соединения;
*       3) принимаемую от сервера строку и время приёма;
*   - разрыв соединения командой: disconnect <адрес> <порт>;
*   - подключение к серверу специальной командой: connect <адрес> <порт>.
*
* Окно (Swing): сверху журнал обмена, снизу строка ввода. Если графики нет
* (или запуск с ключом --console), работает консольный режим: строка уходит по Enter.
* Запуск: java TCPClient
*/
public class TCPClient {

    private final Consumer<String> ui;                // куда выводить сообщения
пользователю
    private final PrintWriter logFile;                // файл протокола
    private final SimpleDateFormat fmt = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss.SSS");

    private Socket socket;
    private BufferedReader in;
    private Writer out;
    private String host;
    private int port;
    private volatile boolean connected = false;

    public TCPClient(Consumer<String> ui) throws IOException {
        this.ui = ui;
        this.logFile = new PrintWriter(new OutputStreamWriter(
            new FileOutputStream("tcp_client.log", true), StandardCharsets.UTF_8),
true);
    }

    /** Подключение к серверу и запуск потока приёма. */
    public synchronized void connect(String h, int p) {
        try {
            socket = new Socket(InetAddress.getByName(h), p);
        } catch (IOException e) {
            ui.accept("Не удалось подключиться к " + h + ":" + p + " (" + e.getMessage()
+ ")");
            return;
        }
        try {
            in = new BufferedReader(new InputStreamReader(socket.getInputStream(),
StandardCharsets.UTF_8));
            out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream(),
StandardCharsets.UTF_8));
        } catch (IOException e) {
            ui.accept("Ошибка потоков: " + e.getMessage());
            return;
        }
        host = socket.getInetAddress().getHostAddress();
        port = p;
        connected = true;
        log("СОЕДИНЕНИЕ НАЧАТО", host + ":" + port);
        ui.accept("Соединение с " + host + ":" + port);
        Thread receiver = new Thread(this::receiveLoop, "receiver");
        receiver.setDaemon(true);
        receiver.start();
    }

    /** Обработка строки, введённой пользователем. */
    public synchronized void onInput(String line) {
        line = line.trim();
        if (line.isEmpty()) return;

        // Специальная команда подключения: connect <адрес> <порт>
        if (line.startsWith("connect")) {
            String[] p = line.split("\\s+");
            if (p.length != 3) {
                ui.accept("Формат команды: connect <адрес> <порт>");
            }
        }
    }
}

```

```

        return;
    }
    try {
        if (connected) closeSession("переподключение");
        connect(p[1], Integer.parseInt(p[2]));
    } catch (NumberFormatException e) {
        ui.accept("Порт должен быть числом.");
    }
    return;
}
// Команда разрыва соединения: disconnect <адрес> <порт>
if (line.startsWith("disconnect")) {
    String[] p = line.split("\\s+");
    if (p.length != 3) {
        ui.accept("Формат команды: disconnect <адрес> <порт>");
        return;
    }
    disconnect(p[1], p[2]);
    return;
}
if (!connected) {
    ui.accept("Нет соединения. Введите: connect <адрес> <порт>");
    return;
}
try {
    out.write(line + "\r\n");
    out.flush(); // без сброса буфера строка не уйдёт в
сеть
    ui.accept("Вы: " + line);
} catch (IOException e) {
    ui.accept("Ошибка отправки: " + e.getMessage());
}
}

/** Разрыв соединения по команде disconnect <адрес> <порт>. */
private void disconnect(String h, String portText) {
    if (!connected) {
        ui.accept("Нет соединения.");
        return;
    }
    try {
        boolean same = InetAddress.getByName(h).getHostAddress().equals(host)
            && Integer.parseInt(portText) == port;
        if (!same) {
            ui.accept("Активное соединение: " + host + ":" + port);
            return;
        }
        closeSession("команда disconnect");
        ui.accept("Соединение разорвано.");
    } catch (NumberFormatException | IOException e) {
        ui.accept("Неверные параметры команды disconnect.");
    }
}

/** Цикл приёма строк от сервера (отдельный поток, чтобы окно не зависало). */
private void receiveLoop() {
    final Socket mySocket = socket; // сокет, который обслуживает
этот поток
    final BufferedReader myIn = in;
    try {
        String s;
        while ((s = myIn.readLine()) != null) { // null - сервер закрыл
соединение
            if (!s.isEmpty()) {
                log("ПОЛУЧЕНО", s); // п. 3 задания: принятая
строка и время
                ui.accept("Сервер: " + s);
            }
        }
    } catch (IOException e) {
        // сокет закрыт нами или оборвалась связь
    }
}

```

```

        synchronized (this) {
            if (connected && socket == mySocket) {          // не трогаем более новое
соединение
                closeSession("закрыто сервером");
                ui.accept("Сервер закрыл соединение.");
            }
        }

    /** Завершение сеанса: закрытие сокета и запись времени окончания в протокол. */
    private void closeSession(String reason) {
        connected = false;
        try { socket.close(); } catch (IOException ignored) { }
        log("СОЕДИНЕНИЕ ЗАВЕРШЕНО", host + ":" + port + " (" + reason + ")");
    }

    public synchronized void shutdown() {
        if (connected) closeSession("выход из программы");
        logFile.close();
    }

    /** Строка протокола: время | событие | данные. */
    private synchronized void log(String event, String data) {
        logFile.println(fmt.format(new Date()) + " | " + event + " | "
            + data.replace("\r", "").replace("\n", "\\n"));
    }

    // ----- запуск

    public static void main(String[] args) throws Exception {
        boolean console = GraphicsEnvironment.isHeadless()
            || (args.length > 0 && args[0].equals("--console"));
        if (console) runConsole(); else runWindow();
    }

    /** Консольный режим: строка уходит по Enter. */
    private static void runConsole() throws IOException {
        TCPClient client = new TCPClient(System.out::println);
        System.out.println("ожидание... Введите: connect <адрес> <порт>");
        BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in));
        String line;
        while ((line = stdIn.readLine()) != null) client.onInput(line);
        client.shutdown();
    }

    /** Оконный режим: строка уходит по клавише Home. */
    private static void runWindow() throws Exception {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("TCP-клиент (отправка строки - клавиша Home)");
            JTextArea area = new JTextArea();
            area.setEditable(false);
            area.setLineWrap(true);
            area.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 14));
            JTextField input = new JTextField();
            input.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 14));

            final TCPClient c;
            try {
                c = new TCPClient(msg -> SwingUtilities.invokeLater(() -> {
                    area.append(msg + "\n");
                    area.setCaretPosition(area.getDocument().getLength());
                }));
            } catch (IOException e) {
                JOptionPane.showMessageDialog(null, "Ошибка: " + e.getMessage());
                return;
            }

            // Отправка строки по нажатию клавиши Home.
            input.addKeyListener(new KeyAdapter() {
                @Override
                public void keyPressed(KeyEvent e) {
                    if (e.getKeyCode() == KeyEvent.VK_HOME) {
                        e.consume();          // не двигаем курсор

```

```

        String text = input.getText();
        input.setText("");
        new Thread(() -> c.onInput(text)).start();
    }
}
});
frame.addWindowListener(new WindowAdapter() {
    @Override
    public void windowClosing(WindowEvent e) {
        c.shutdown();
        frame.dispose();
        System.exit(0);
    }
});
frame.setDefaultCloseOperation(WindowConstants.DO_NOTHING_ON_CLOSE);
frame.add(new JScrollPane(area), BorderLayout.CENTER);
frame.add(input, BorderLayout.SOUTH);
frame.setSize(700, 450);
frame.setLocationRelativeTo(null);
frame.setVisible(true);
area.append("ожидание...\nПодключение: connect 127.0.0.1 8080\n"
    + "Разрыв: disconnect 127.0.0.1 8080\n"
    + "Отправка введённой строки - клавиша Home.\n");
input.requestFocusInWindow();
});
}
}

```

Результаты работы:

Проверка выполнена с TCP-сервером из лабораторной работы №1. Последовательность: подключение командой connect; отправка строки «hello tcp»; отправка строки, дополняющей группу до 64 символов (сервер присылает статистику); команда disconnect.

```
ожидание...
Подключение: connect 127.0.0.1 8080
Разрыв: disconnect 127.0.0.1 8080
Отправка введенной строки - клавиша Home.
Соединение с 127.0.0.1:8080
Сервер: Hello, Student!
Вы: hello tcp
Сервер: hello tcp
Вы: the quick brown fox jumps over the lazy dog and runs fa
Сервер: the quick brown fox jumps over the lazy dog and runs fa
Сервер: <'h'-3, 'e'-4, 'l'-3, 'o'-5, ' '-12, 't'-3, 'c'-2, 'p'-2, 'q'-1, 'u'-3, 'i'-1,
'k'-1, 'b'-1, 'r'-3, 'w'-1, 'n'-3, 'f'-2, 'x'-1, 'j'-1, 'm'-1, 's'-2, 'v'-1, 'a'-3, 'z'-1, 'y'-1, 'd'-2, 'g'-1>
Соединение разорвано.
```

Рисунок 1 – Окно TCP-клиента

```
2026-10-04 23:48:01.088 | СОЕДИНЕНИЕ НАЧАТО | 127.0.0.1:8080
2026-10-04 23:48:01.112 | ПОЛУЧЕНО | Hello, Student!
2026-10-04 23:48:02.021 | ПОЛУЧЕНО | hello tcp
2026-10-04 23:48:04.207 | ПОЛУЧЕНО | the quick brown fox jumps over the lazy dog and runs fa
2026-10-04 23:48:04.207 | ПОЛУЧЕНО | <'h'-3, 'e'-4, 'l'-3, 'o'-5, ' '-12, 't'-3, 'c'-2, 'p'-2,
'q'-1, 'u'-3, 'i'-1, 'k'-1, 'b'-1, 'r'-3, 'w'-1, 'n'-3, 'f'-2, 'x'-1, 'j'-1, 'm'-1, 's'-2,
'v'-1, 'a'-3, 'z'-1, 'y'-1, 'd'-2, 'g'-1>
2026-10-04 23:48:05.574 | СОЕДИНЕНИЕ ЗАВЕРШЕНО | 127.0.0.1:8080 (команда disconnect)
```

Рисунок 2 – Файл протокола событий tcp_client.log

```
23:47:55.819 Сервер запущен. Порт 8080
23:48:01.111 Подключился клиент 127.0.0.1:46654
23:48:04.202 Группа от 127.0.0.1:46654: <'h'-3, 'e'-4, 'l'-3, 'o'-5, ' '-12, 't'-3, 'c'-2,
'p'-2, 'q'-1, 'u'-3, 'i'-1, 'k'-1, 'b'-1, 'r'-3, 'w'-1, 'n'-3, 'f'-2, 'x'-1, 'j'-1, 'm'-1,
's'-2, 'v'-1, 'a'-3, 'z'-1, 'y'-1, 'd'-2, 'g'-1>
23:48:05.567 Соединение с 127.0.0.1:46654 закрыто
```

Рисунок 3 – Консоль TCP-сервера

Вывод: В ходе работы разработан TCP-клиент: подключение командой connect, отправка строки по клавише Home, разрыв соединения командой disconnect, ведение файла протокола с временем начала и окончания соединения и принятыми от сервера строками. Совместная работа с сервером из лабораторной работы №1 проверена на 127.0.0.1:8080.

Лабораторная работа №3. Организация UDP-сервера и UDP-клиента

Цель работы: Изучить протокол UDP и классы DatagramSocket и DatagramPacket пакета java.net; разработать UDP-сервер и UDP-клиент в соответствии с индивидуальным заданием (вариант 4).

Индивидуальное задание (вариант 4):

Сервер (задание №4)	После приёма каждой группы из 64 символов от клиента сервер формирует и отправляет цепочку <«символ1-количество1», «символ2-количество2», ...>, где «символ» – встреченный в группе символ, «количество» – число его вхождений в группе. Если в очередной группе число различных символов меньше 3, клиенту отправляется соответствующее сообщение и сервер разрывает соединение.
Клиент (задание №5)	Ввод символов с отсылкой введенной строки по нажатию клавиши Home. Ведение файла протокола событий: 1) время начала и окончания соединения; 3) принимаемая от сервера строка и время приёма. Разрыв соединения командой disconnect <адрес> <порт>. Подключение к серверу специальной командой connect <адрес> <порт>.
Параметры проверки	Адрес 127.0.0.1, порт 666 (п. 10.4 методических указаний).

Краткие теоретические сведения:

UDP (User Datagram Protocol) – протокол транспортного уровня без установления соединения. В отличие от TCP он не гарантирует доставку датаграмм и их порядок, но работает быстрее из-за малых накладных расходов. Каждая датаграмма самостоятельна и содержит адрес и порт отправителя, поэтому в UDP нет входных и выходных потоков – обмен идёт только пакетами (набором байт).

В пакете java.net для работы с UDP используются два класса. DatagramSocket – сокет, через который датаграммы отправляются (метод send) и принимаются (метод receive; блокирует выполнение, пока не придёт пакет). Сервер создаёт сокет с указанием порта (new DatagramSocket(666)), клиент – без порта, свободный порт выбирает система. DatagramPacket – сама датаграмма: массив байт, его длина и, для исходящего пакета, адрес и порт получателя. Для принятого пакета методы getAddress() и getPort() возвращают адрес и порт отправителя, что позволяет серверу ответить клиенту.

Так как UDP не имеет соединений, «соединение» и «разрыв соединения» из задания реализуются на уровне прикладного протокола: сервер хранит список клиентов (адрес:порт), клиент командой connect запоминает адрес сервера и сообщает ему о подключении, командой disconnect сообщает об отключении.

Ход работы:

Прикладной протокол обмена. Клиент → сервер: служебная датаграмма «connect» (подключение, отправляется при вводе команды connect), служебная «END» (разрыв соединения по команде disconnect), остальные датаграммы – введенный пользователем текст. Сервер → клиент: приветствие, подтверждение приёма неполной группы, цепочка «символ-количество» для каждой полной группы из 64 символов, сообщение с меткой [BYE] при разрыве соединения сервером (метка клиентом не показывается, а служит сигналом закрыть сеанс). Символы от клиента накапливаются в буфере на стороне сервера, поэтому группа из 64 символов может набираться из нескольких строк.

Исправления относительно примера из методических указаний: строка из пакета создаётся только по принятым байтам new String(packet.getData(), 0, packet.getLength()), чтобы в неё не

попадали нулевые байты буфера; в примере сервера не определена переменная result и нарушена структура скобок, код приведён в рабочий вид.

Листинг программы UDPServerThread.java:

```
import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.nio.charset.StandardCharsets;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Лабораторная работа №3. UDP-сервер.
 * Вариант 4: задание сервера №4.
 *
 * Сервер накапливает символы, присланные клиентом. После приёма каждой группы
 * из 64 символов он формирует и отправляет клиенту цепочку
 * <'символ1'-количество1, 'символ2'-количество2, ...>, где "символ" - встреченный
 * в группе символ, а "количество" - сколько раз он встретился в этой группе.
 * Если в очередной группе различных символов меньше 3, сервер отправляет
 * соответствующее сообщение и разрывает "соединение".
 *
 * UDP не имеет соединений, поэтому "соединение" эмулируется таблицей клиентов
 * (ключ "адрес:порт", значение - буфер ещё не обработанных символов).
 */
public class UDPServerThread {

    /** Порт сервера (по заданию лабораторной работы - 666). */
    public static int PORT = 666;

    /** Размер группы символов, по которой строится статистика. */
    static final int GROUP = 64;

    /** Минимально допустимое число различных символов в группе. */
    static final int MIN_DISTINCT = 3;

    /** Метка в начале ответа: сервер закрыл "соединение" с клиентом. */
    static final String BYE = "[BYE]";

    /** Клиенты, с которыми "установлено соединение": "адрес:порт" -> буфер символов. */
    static final Map<String, StringBuilder> sessions = new HashMap<>();

    public static void main(String[] args) throws IOException {
        if (args.length > 0) {
            PORT = Integer.parseInt(args[0]);
        }
        DatagramSocket socket = new DatagramSocket(PORT);
        log("Сервер запущен. Порт " + PORT);

        while (true) {
            try {
                // Буфер под входящую датаграмму.
                byte[] buf = new byte[2048];
                DatagramPacket packet = new DatagramPacket(buf, buf.length);

                // Блокируемся, пока не придёт датаграмма. Пакет "помнит",
                // откуда пришёл: адрес и порт клиента лежат внутри него.
                socket.receive(packet);

                // Берём только реально принятые байты (packet.getLength()),
                // иначе в строку попадёт хвост из нулей буфера.
                String received = new String(packet.getData(), 0,
                    packet.getLength(), StandardCharsets.UTF_8);

                InetAddress address = packet.getAddress();
                int port = packet.getPort();
                String client = address.getHostAddress() + ":" + port;
```

```

        log("<- " + client + " : " + received);

        handle(socket, client, address, port, received);
    } catch (IOException e) {
        e.printStackTrace();
    }
}

/** Разбор одной принятой датаграммы. */
static void handle(DatagramSocket socket, String client,
    InetAddress address, int port, String text) throws IOException {
    // 1. Служебное: клиент подключается.
    if (text.equals("connect")) {
        sessions.put(client, new StringBuilder());
        log("Соединение установлено: " + client);
        send(socket, address, port, "Hello, Student! Присылайте текст: статистика
отправляется после каждых "
            + GROUP + " символов.");
        return;
    }
    // 2. Служебное: клиент разорвал соединение командой disconnect - ответа нет.
    if (text.equals("END")) {
        sessions.remove(client);
        log("Соединение закрыто клиентом: " + client);
        return;
    }
    // 3. Данные принимаются только после connect.
    StringBuilder buffer = sessions.get(client);
    if (buffer == null) {
        send(socket, address, port, BYE + " Соединение не установлено. Выполните
connect.");
        return;
    }
    // 4. Накапливаем символы и обрабатываем каждую полную группу из 64 символов.
    buffer.append(text);
    while (buffer.codePointCount(0, buffer.length()) >= GROUP) {
        int end = buffer.offsetByCodePoints(0, GROUP);
        String group = buffer.substring(0, end);
        buffer.delete(0, end);

        // Считаем, сколько раз встретился каждый символ (порядок - по первому
появлению).
        Map<String, Integer> counts = new LinkedHashMap<>();
        group.codePoints().forEach(cp ->
            counts.merge(new String(Character.toChars(cp)), 1, Integer::sum));

        if (counts.size() < MIN_DISTINCT) {
            // Разных символов меньше 3: сообщаем и разрываем соединение.
            sessions.remove(client);
            log("В группе только " + counts.size() + " различных символов.
Соединение с "
                + client + " разорвано");
            send(socket, address, port, BYE + " В группе из " + GROUP + " символов
меньше "
                + MIN_DISTINCT + " различных (найдено " + counts.size()
                + "). Соединение разорвано.");
            return;
        }
        // Формируем цепочку <'a'-5, 'b'-3, ...>.
        StringBuilder chain = new StringBuilder("<");
        for (Map.Entry<String, Integer> e : counts.entrySet()) {
            if (chain.length() > 1) chain.append(", ");
            chain.append('\'' + e.getKey() + '-' + e.getValue() + "'");
        }
        chain.append('>');
        send(socket, address, port, chain.toString());
    }
    // 5. Подтверждение приёма неполной группы (чтобы клиент видел, что сервер жив).
    int left = buffer.codePointCount(0, buffer.length());
    if (left > 0) {
        send(socket, address, port, "Принято. В буфере " + left + " из " + GROUP + "

```

```

СИМВОЛОВ.");
    }
}

/** Отправка строки клиенту в виде датаграммы на его адрес и порт. */
static void send(DatagramSocket socket, InetAddress address, int port, String text)
    throws IOException {
    byte[] data = text.getBytes(StandardCharsets.UTF_8);
    DatagramPacket reply = new DatagramPacket(data, data.length, address, port);
    socket.send(reply);
    log("-> " + address.getHostAddress() + ":" + port + " : " + text);
}

static void log(String msg) {
    System.out.println(new SimpleDateFormat("HH:mm:ss.SSS").format(new Date()) + "
" + msg);
}
}

```

Листинг программы UDPClient.java:

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import java.io.*;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.nio.charset.StandardCharsets;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.function.Consumer;

/**
 * Лабораторная работа №3. UDP-клиент.
 * Вариант 4: задание клиента №5 (табл. 3):
 * - ввод символов с отправкой введённой строки по нажатию клавиши Home;
 * - ведение файла протокола событий client.log, включающего:
 *     1) время начала и окончания соединения;
 *     3) принимаемую от сервера строку и время приёма;
 * - разрыв соединения командой: disconnect <адрес> <порт>;
 * - подключение к серверу специальной командой: connect <адрес> <порт>.
 *
 * Окно (Swing): сверху - журнал обмена, снизу - строка ввода.
 * Если графики нет (или запуск с ключом --console), работает консольный режим,
 * где строка отправляется по Enter (консоль не умеет ловить клавишу Home).
 */
public class UDPClient {

    /** Порт сервера, к которому подключён клиент (заполняется командой connect). */
    public static int PORT;

    /** Метка сервера: "соединение" закрыто сервером. */
    static final String BYE = "[BYE]";

    private final DatagramSocket socket; // один сокет на всё время работы
    private final Consumer<String> ui; // куда выводить сообщения пользователю
    private final PrintWriter logFile; // файл протокола
    private final SimpleDateFormat fmt = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss.SSS");

    private InetAddress serverAddress; // адрес сервера (после connect)
    private volatile boolean connected = false;

    public UDPClient(Consumer<String> ui) throws IOException {
        this.ui = ui;
        this.socket = new DatagramSocket(); // свободный локальный порт выберет
система

```

```

        this.logFile = new PrintWriter(new OutputStreamWriter(
            new FileOutputStream("client.log", true), StandardCharsets.UTF_8),
true);
    // Поток приёма: ждёт датаграммы от сервера независимо от ввода пользователя.
    Thread receiver = new Thread(this::receiveLoop, "receiver");
    receiver.setDaemon(true);
    receiver.start();
}

/** Обработка строки, введённой пользователем. */
public synchronized void onInput(String line) {
    line = line.trim();
    if (line.isEmpty()) return;

    // Специальная команда подключения: connect <адрес> <порт>
    if (line.startsWith("connect")) {
        String[] p = line.split("\\s+");
        if (p.length != 3) {
            ui.accept("Формат команды: connect <адрес> <порт>");
            return;
        }
        try {
            connectTo(p[1], Integer.parseInt(p[2]));
        } catch (NumberFormatException e) {
            ui.accept("Порт должен быть числом от 1 до 65535.");
        } catch (IOException e) {
            ui.accept("Не удалось подключиться: " + e.getMessage());
        }
        return;
    }
    // Команда разрыва соединения: disconnect <адрес> <порт>
    if (line.startsWith("disconnect")) {
        String[] p = line.split("\\s+");
        if (p.length != 3) {
            ui.accept("Формат команды: disconnect <адрес> <порт>");
            return;
        }
        disconnectFrom(p[1], p[2]);
        return;
    }
    if (!connected) {
        ui.accept("Нет соединения. Введите: connect <адрес> <порт>");
        return;
    }
    try {
        send(line);
    } catch (IOException e) {
        ui.accept("Ошибка отправки: " + e.getMessage());
    }
}

/** "Подключение": запоминаем адрес сервера и отправляем приветственную датаграмму.
*/
private void connectTo(String host, int port) throws IOException {
    if (port < 1 || port > 65535) throw new NumberFormatException();
    if (connected) closeSession("переподключение");
    serverAddress = InetAddress.getByName(host);
    PORT = port;
    connected = true;
    log("СОЕДИНЕНИЕ НАЧАТО", serverAddress.getHostAddress() + ":" + PORT);
    ui.accept("Соединение с " + serverAddress.getHostAddress() + ":" + PORT);
    rawSend("connect");
}

/** Разрыв соединения: сообщаем серверу (END) и пишем время окончания в протокол. */
private void disconnectFrom(String host, String portText) {
    if (!connected) {
        ui.accept("Нет соединения.");
        return;
    }
    try {
        boolean same = InetAddress.getByName(host).equals(serverAddress)

```

```

        && Integer.parseInt(portText) == PORT;
        if (!same) {
            ui.accept("Активное соединение: " + serverAddress.getHostAddress() + ":" +
+ PORT);
            return;
        }
        rawSend("END");
        closeSession("команда disconnect");
        ui.accept("Соединение разорвано.");
    } catch (NumberFormatException | IOException e) {
        ui.accept("Неверные параметры команды disconnect.");
    }
}

/** Отправка строки серверу (в протокол не пишется - по заданию логируются только п.
1 и 3). */
private void send(String text) throws IOException {
    rawSend(text);
    ui.accept("Вы: " + text);
}

private void rawSend(String text) throws IOException {
    byte[] data = text.getBytes(StandardCharsets.UTF_8);
    if (data.length > 1000) throw new IOException("слишком длинная строка (больше
1000 байт)");
    DatagramPacket datagram = new DatagramPacket(data, data.length, serverAddress,
PORT);
    socket.send(datagram);
}

/** Цикл приёма датаграмм от сервера (работает в отдельном потоке). */
private void receiveLoop() {
    while (!socket.isClosed()) {
        try {
            byte[] buf = new byte[2048];
            DatagramPacket datagram = new DatagramPacket(buf, buf.length);
            socket.receive(datagram); // ждём ответ сервера
            // Используем только принятые байты (getLength), а не весь буфер.
            String received = new String(datagram.getData(), 0,
                datagram.getLength(), StandardCharsets.UTF_8);
            boolean bye = received.startsWith(BYE);
            if (bye) received = received.substring(BYE.length()).trim();
            log("ПОЛУЧЕНО", received);
            ui.accept("Сервер: " + received);
            if (bye) {
                synchronized (this) {
                    if (connected) closeSession("закрыто сервером");
                }
                ui.accept("Соединение закрыто. Для работы введите: connect <адрес>
<порт>");
            }
        } catch (IOException e) {
            if (!socket.isClosed()) ui.accept("Ошибка приёма: " + e.getMessage());
        }
    }
}

/** Завершение сеанса: запись времени окончания соединения в протокол. */
private void closeSession(String reason) {
    connected = false;
    log("СОЕДИНЕНИЕ ЗАВЕРШЕНО", serverAddress.getHostAddress() + ":" + PORT + " (" +
reason + ")");
}

/** Корректное закрытие клиента. */
public synchronized void shutdown() {
    if (connected) {
        try { rawSend("END"); } catch (IOException ignored) { }
        closeSession("выход из программы");
    }
    socket.close();
    logFile.close();
}

```

```

    }

    /** Строка протокола: время | событие | данные. */
    private synchronized void log(String event, String data) {
        logFile.println(fmt.format(new Date()) + " | " + event + " | "
            + data.replace("\r", "").replace("\n", "\\n"));
    }

    // ----- запуск

    public static void main(String[] args) throws Exception {
        boolean console = GraphicsEnvironment.isHeadless()
            || (args.length > 0 && args[0].equals("--console"));
        if (console) runConsole(); else runWindow();
    }

    /** Консольный режим: строка уходит по Enter. */
    private static void runConsole() throws IOException {
        UDPClient client = new UDPClient(msg -> System.out.println(msg));
        System.out.println("ожидание... Введите: connect <адрес> <порт>");
        BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in));
        String line;
        while ((line = stdIn.readLine()) != null) {
            client.onInput(line);
        }
        client.shutdown();
    }

    /** Оконный режим: строка уходит по клавише Home. */
    private static void runWindow() throws Exception {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("UDP-клиент (отправка строки - клавиша Home)");
            JTextArea area = new JTextArea();
            area.setEditable(false);
            area.setLineWrap(true);
            area.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 14));
            JTextField input = new JTextField();
            input.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 14));

            UDPClient client;
            try {
                client = new UDPClient(msg -> SwingUtilities.invokeLater(() -> {
                    area.append(msg + "\n");
                    area.setCaretPosition(area.getDocument().getLength());
                }));
            } catch (IOException e) {
                JOptionPane.showMessageDialog(null, "Ошибка: " + e.getMessage());
                return;
            }
            final UDPClient c = client;

            // Отправка строки по нажатию клавиши Home.
            input.addKeyListener(new KeyAdapter() {
                @Override
                public void keyPressed(KeyEvent e) {
                    if (e.getKeyCode() == KeyEvent.VK_HOME) {
                        e.consume(); // не двигаем курсор
                        String text = input.getText();
                        input.setText("");
                        // отправку делаем вне потока интерфейса, чтобы окно не
                        "зависало"
                        new Thread(() -> c.onInput(text)).start();
                    }
                }
            });
            frame.addWindowListener(new WindowAdapter() {
                @Override
                public void windowClosing(WindowEvent e) {
                    c.shutdown();
                    frame.dispose();
                    System.exit(0);
                }
            });
        });
    }

```

```
});
frame.setDefaultCloseOperation(WindowConstants.DO_NOTHING_ON_CLOSE);
frame.add(new JScrollPane(area), BorderLayout.CENTER);
frame.add(input, BorderLayout.SOUTH);
frame.setSize(700, 450);
frame.setLocationRelativeTo(null);
frame.setVisible(true);
area.append("ожидание...\nПодключение: connect 127.0.0.1 666\n"
           + "Разрыв: disconnect 127.0.0.1 666\n"
           + "Отправка введённой строки - клавиша Home.\n");
input.requestFocusInWindow();
});
}
}
```

Для проверки сервер и клиент запущены на одном компьютере, адрес 127.0.0.1, порт 666. Последовательность: подключение командой connect; короткая строка «hello udr» (в буфере 9 из 64 символов); строка, дополняющая группу до 64 символов (сервер возвращает статистику символов); группа из 64 символов с двумя различными символами (сервер разрывает соединение); попытка отправки без соединения; повторное подключение и разрыв командой disconnect.

Рисунок 1 – Окно UDP-клиента

Рисунок 2 – Консоль UDP-сервера

```
2026-10-04 16:19:56.945 | СОЕДИНЕНИЕ НАЧАТО | 127.0.0.1:666
2026-10-04 16:19:56.957 | ПОЛУЧЕНО | Hello, Student! Присылайте текст: статистика отправляется
после каждых 64 символов.
2026-10-04 16:19:57.927 | ПОЛУЧЕНО | Принято. В буфере 9 из 64 символов.
2026-10-04 16:20:00.328 | ПОЛУЧЕНО | <'h'-3, 'e'-4, 'l'-3, 'o'-5, ' ' -12, 'u'-4, 'd'-3, 'p'-2,
't'-2, 'q'-1, 'i'-1, 'c'-1, 'k'-1, 'b'-1, 'r'-3, 'w'-1, 'n'-3, 'f'-2, 'x'-1, 'j'-1, 'm'-1,
's'-2, 'v'-1, 'a'-3, 'z'-1, 'y'-1, 'q'-1>
```

```
2026-10-04 16:20:02.996 | ПОЛУЧЕНО | В группе из 64 символов меньше 3 различных (найдено 2).  
Соединение разорвано.  
2026-10-04 16:20:03.005 | СОЕДИНЕНИЕ ЗАВЕРШЕНО | 127.0.0.1:666 (закрыто сервером)  
2026-10-04 16:20:05.176 | СОЕДИНЕНИЕ НАЧАТО | 127.0.0.1:666  
2026-10-04 16:20:05.178 | ПОЛУЧЕНО | Hello, Student! Присылайте текст: статистика отправляется  
после каждые 64 символов.  
2026-10-04 16:20:06.609 | СОЕДИНЕНИЕ ЗАВЕРШЕНО | 127.0.0.1:666 (команда disconnect)
```

Рисунок 3 – Файл протокола событий client.log

Вывод: В ходе работы изучены протокол UDP и классы DatagramSocket и DatagramPacket. Разработаны UDP-сервер, строящий статистику символов по каждой группе из 64 символов и разрывающий соединение при числе различных символов меньше 3, и UDP-клиент с командами connect и disconnect, отправкой строки по клавише Home и файлом протокола. Проверка на 127.0.0.1:666 подтвердила корректную работу. Особенность UDP – отсутствие соединений и гарантий доставки, поэтому понятия соединения и его разрыва реализованы средствами прикладного протокола.